

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Organización de Lenguajes y Compiladores 2
Primer Semestre 2025

Catedráticos:

Ing. Bayron López

Ing. Edgar Saban

Ing. Luis Espino

Tutores académicos:

Henry Mendoza

Ronaldo Posadas

Damián Peñal



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

GoLight Proyecto 1



Indice

Objetivo general.....	3
Objetivos específicos.....	3
Requisitos mínimos.....	4



1. Objetivos

1.1 Objetivos Generales

Capacidad para desarrollar un interprete funcional para un lenguaje de programacion aplicando los principios fundamentales de la teoria de compiladores, utilizando:

- Análisis Léxico: Generación de un analizador léxico para la identificación y clasificación de tokens.
- Análisis Sintáctico: Implementación de un analizador sintáctico que valide la estructura gramatical del código.
- Análisis Semántico: Aplicación de reglas semánticas para garantizar la coherencia del lenguaje.
- Generación y Recorrido del AST: Construcción y manipulación eficiente del Árbol de Sintaxis Abstracta (AST) para la correcta interpretación del código.

1.2 Objetivos específicos

- Uso de ANTLR: Implementación de analizadores léxicos y sintácticos mediante ANTLR en un entorno de desarrollo basado en C#.
- Estructuras y Herramientas Adecuadas: Selección e implementación de estructuras de datos y herramientas apropiadas para la construcción del intérprete.
- Manejo del AST: Desarrollo de técnicas de recorrido del Árbol de Sintaxis Abstracta (AST) para la generación de estructuras fundamentales como la tabla de símbolos, asegurando una interpretación y ejecución correcta del lenguaje.

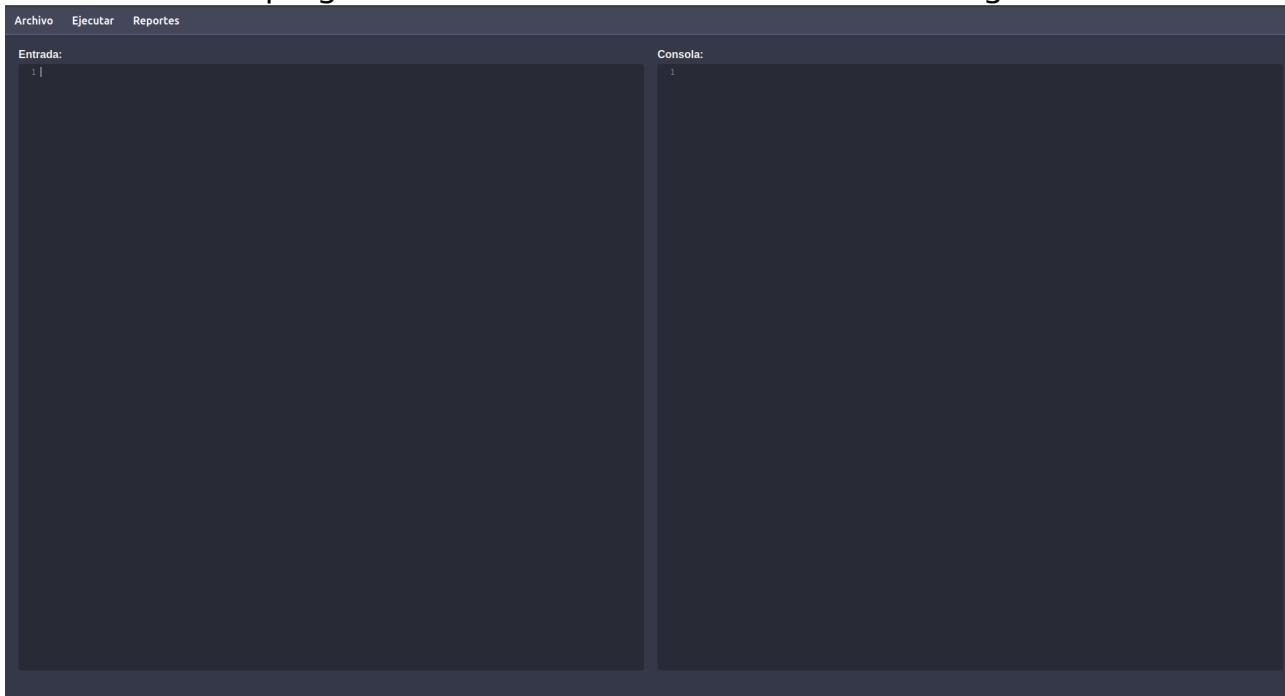
Requisitos mínimos

Para el funcionamiento óptimo del programa se requiere:

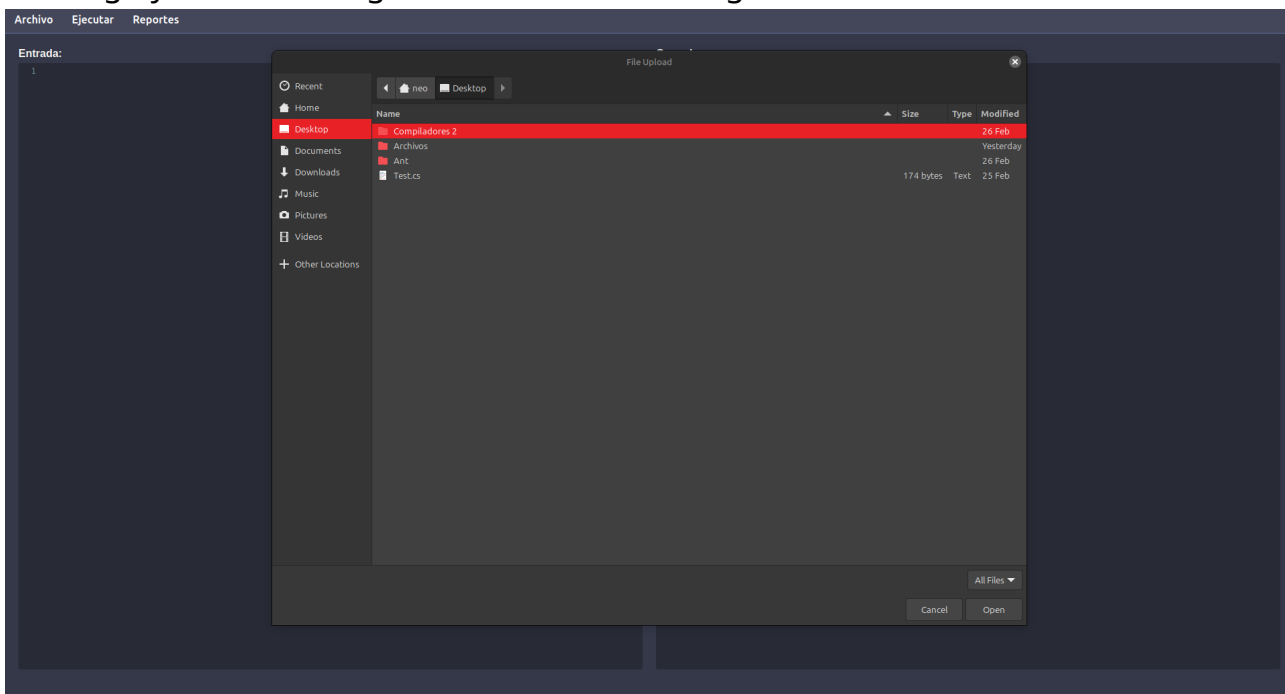
- Espacio en disco mínimo de 500 mb
- Memoria RAM de 2 GB
- Poseer C# instalado en la maquina, junto a ANTLR4.

Utilización del programa

Al iniciar el programa nos encontraremos con esta interfaz gráfica.



Al ingresar en archivo, opción abrir, podremos cargar un archivo de extensión .glt y este será cargado en consola de la siguiente manera:



Tras esto, el botón ejecutar nos permitirá realizar el análisis tanto léxico como sintáctico del programa, permitiendo la lectura del archivo y la actualización en consola.

ArchivoEjecutarReportes

Entrada:

1 puntos := 0
2
3 fmt.Println("==== Archivo de prueba básico ===")
4
5 // 1. Declaración de variables (2 puntos)
6 fmt.Println("==== Declaración de variables ===")
7 puntosDeclaracion := 0
8
9 fmt.Println("Declaración explícita con tipo y valor")
10 var entero int = 42
11 var decimal float64 = 3.14159
12 var texto string = "Hola, mundo!"
13 var booleano bool = true
14
15 fmt.Println("entero:", entero)
16 fmt.Println("decimal:", decimal)
17 fmt.Println("texto:", texto)
18 fmt.Println("booleano:", booleano)
19
20 if entero == 42 && decimal > 3.0 && texto == "Hola, mundo!" && booleano == true {
21 puntosDeclaracion = puntosDeclaracion + 1
22 fmt.Println("OK Declaración explícita con tipo y valor: correcto")
23 } else {
24 fmt.Println("X Declaración explícita con tipo y valor: incorrecto")
25 }
26
27 fmt.Println("\ndeclaración explícita con tipo y sin valor")
28 var enteroSinValor int
29 var decimalSinValor float64
30 var textoSinValor string
31 var booleanoSinValor bool
32
33 fmt.Println("enteroSinValor:", enteroSinValor)
34 fmt.Println("decimalSinValor:", decimalSinValor)
35 fmt.Println("textoSinValor:", textoSinValor)
36 fmt.Println("booleanoSinValor:", booleanoSinValor)
37
38 // Verificamos que se inicialicen con valores por defecto
39 if enteroSinValor == 0 && decimalSinValor == 0.0 && textoSinValor == "" && booleanoSinValor == false {
40 puntosDeclaracion = puntosDeclaracion + 1
41 fmt.Println("OK Declaración explícita con tipo y sin valor: correcto")
42 } else {
43 fmt.Println("X Declaración explícita con tipo y sin valor: incorrecto")
44 }
45
46 fmt.Println("\nErrores de redeclaración")
47 // var entero int = 100 // ! ERROR: Comentar esta línea para que el programa compile
48 if entero == 100 {
49 puntosDeclaracion = puntosDeclaracion - 1
50 fmt.Println("X Errores de redeclaración: incorrecto")
51 } else {
52 fmt.Println("OK Errores de redeclaración: correcto")
53 }
54
55 // 2. Asignación de variables (2 puntos)
56 fmt.Println("\n==== Asignación de variables ===")
57 puntosAsignacion := 0
58
59 fmt.Println("Asignación con tipo correcto")
60 entero = 99

Consola:

1 === Archivo de prueba básico ===
2 === Declaración de variables ===
3 Declaración explícita con tipo y valor
4 entero: 42
5 decimal: 3.14
6 texto: Hola, mundo!
7 booleano: true
8 OK Declaración explícita con tipo y valor: correcto
9
10 Declaración explícita con tipo y sin valor
11 enteroSinValor: 0
12 decimalSinValor: 0.00
13 textoSinValor:
14 booleanoSinValor: False
15 OK Declaración explícita con tipo y sin valor: correcto
16
17 Errores de redeclaración
18 OK Errores de redeclaración: correcto
19
20 === Asignación de variables ===
21 Asignación con tipo correcto
22 entero: 99
23 decimal: 9.99
24 texto: Texto modificado
25 booleano: False
26 OK Asignación con tipo correcto: correcto
27
28 Asignación con tipo incorrecto
29 OK Asignación con tipo incorrecto: Se detectaron errores de tipo correctamente
30
31 === Operaciones Aritméticas ===
32 Suma
33 10 + 5 = 15
34 10.5 + 5.5 = 16.00
35 10 + 5.5 = 15.50
36 10.5 + 5 = 15.50
37 OK Suma: correcto
38
39 Resta
40 10 - 5 = 5
41 10.5 - 5.5 = 5.00
42 10 - 5.5 = 4.50
43 10.5 - 5 = 5.50
44 OK Resta: correcto
45
46 === Operaciones Relacionales ===
47 Igualdad
48 10 == 10: true
49 10 == 5: false
50 10.5 == 10.5: true
51 10.5 == 5.5: false
52 OK Igualdad: correcto
53
54 Desigualdad
55 10 != 10: false
56 10 != 5: true
57 10.5 != 10.5: false
58 10.5 != 5.5: true
59 OK Desigualdad: correcto
60

Al hacer click en el botón reportes nos permite elegir sobre que tipo de reporte queremos generar.

ArchivoEjecutarReportes

Reporte de errores

Generar árbol de análisis sintáctico

Generar tabla de símbolos

1 puntos := 0
2
3 fmt.Println("==== Archivo de prueba básico ===")
4
5 // 1. Declaración de variables (2 puntos)
6 fmt.Println("==== Declaración de variables ===")
7 puntosDeclaracion := 0
8
9 fmt.Println("Declaración explícita con tipo y valor")
10 var entero int = 42
11 var decimal float64 = 3.14159
12 var texto string = "Hola, mundo!"
13 var booleano bool = true
14
15 fmt.Println("entero:", entero)
16 fmt.Println("decimal:", decimal)
17 fmt.Println("texto:", texto)
18 fmt.Println("booleano:", booleano)
19
20 if entero == 42 && decimal > 3.0 && texto == "Hola, mundo!" && booleano == true {
21 puntosDeclaracion = puntosDeclaracion + 1
22 fmt.Println("OK Declaración explícita con tipo y valor: correcto")
23 } else {
24 fmt.Println("X Declaración explícita con tipo y valor: incorrecto")
25 }
26
27 fmt.Println("\ndeclaración explícita con tipo y sin valor")

Consola:

1 === Archivo de prueba básico ===
2 === Declaración de variables ===
3 Declaración explícita con tipo y valor
4 entero: 42
5 decimal: 3.14
6 texto: Hola, mundo!
7 booleano: true
8 OK Declaración explícita con tipo y valor: correcto
9
10 Declaración explícita con tipo y sin valor
11 enteroSinValor: 0
12 decimalSinValor: 0.00
13 textoSinValor:
14 booleanoSinValor: False
15 OK Declaración explícita con tipo y sin valor: correcto
16
17 Errores de redeclaración
18 OK Errores de redeclaración: correcto
19
20 === Asignación de variables ===
21 Asignación con tipo correcto
22 entero: 99
23 decimal: 9.99
24 texto: Texto modificado
25 booleano: False
26 OK Asignación con tipo correcto: correcto
27

Al hacer click en reporte de errores, se nos muestran los errores de la siguiente manera.

Tabla de errores

No.	Descripción	Línea	Columna	Tipo
1	El struct no tiene un método llamado 'ObtenerNombre'	197	19	Semántico
2	La variable nombreObtenido no ha sido encontrada	198	33	Semántico
3	La variable nombreObtenido no ha sido encontrada	200	4	Semántico
4	El struct no tiene un método llamado 'CambiarValor'	208	1	Semántico
5	El struct no tiene un método llamado 'ImprimirNombre'	219	1	Semántico
6	El struct no tiene un método llamado 'ObtenerSiguienteNombre'	223	20	Semántico
7	La variable nombreSiguiente no ha sido encontrada	224	43	Semántico
8	El struct no tiene un método llamado 'ObtenerSiguienteNombre'	226	27	Semántico
9	La variable nombreSiguienteDeNodo3 no ha sido encontrada	227	52	Semántico

Esta es una muestra del árbol AST.

